VEME - Milestone 2 Report

Emanuele Nevali | 358702 | emanuele.nevali@epfl.ch
Vincenzo SIgillò Massara | 388059 | vincenzo.sigillomassara@epfl.ch
Emanuele Rimoldi | 377013 | emanuele.rimoldi@epfl.ch
Mattia Moro | 395819 | mattia.moro@epfl.ch
VEME

1 Introduction

This report outlines the current progress of our MNLP project focused on adapting and evaluating four models: DPO, MCQA, Quantized, and RAG. We summarize the data used, training procedures, preliminary results, and next steps for completing the project.

2 Data

2.1 DPO

The dataset combines two sources of preference pairs. The first consists of 24,240 triplets derived from collective student annotations, where `question_body`, `A`, and `B` are mapped to `prompt`, `chosen`, and `rejected` based on the `overall` score, with 18 annotated pairs per question. The second source is a STEM-only subset of StackExchange (Matas et al., 2021), where the most upvoted answer is selected as `chosen`, and a comparable `rejected` answer is generated via a GPT wrapper, yielding 16,000 triplets (bounded by token availability).

2.2 MCQA and Quantization

We address these two models together, since we used the same data for training both of them. In particular, quantization usually does not involve training, but as we will explain later, we tried different approaches among which QLoRa (Dettmers et al., 2023) that involves finetuning and SmoothQuant (Xiao et al., 2024) that needs calibration data.

For finetuning the MCQA model and QLoRa we used a mixture of training data: MathQA (Amini et al., 2019) a large-scale dataset of math word problems, SciQ (Welbl et al., 2017) containing exam questions about Physics, Chemistry and Biology, among others, and GPQA (Rein et al., 2023) a dataset of multiple-choice questions written by domain experts in biology, physics, and chemistry.

For the calibration data we used a subset of 512 selected samples.

2.3 RAG

Our RAG knowledge base was derived from the full Wikipedia dump (`wikimedia/wikipedia` (Foundation)), filtered for STEM topics in Mathematics, Computer Science, Physics, Engineering and Chemistry and split into 800 token segments with 200 token overlap, resulting in 2.25 million entries. We evaluated embedder selection and adapter fine tuning on three held out STEM QA datasets: *MathQA* (Amini et al., 2019), *SciQ* (Welbl et al., 2017) and *Camel AI Math* (Li et al., 2023), using each question as a query and its solution paragraph as ground truth. For triplet loss training, negatives were drawn from solution paragraphs of other questions so that each item served once as an anchor and once as a negative; we also used semi hard negative mining by choosing negatives with moderate cosine similarity to foster a more discriminative embedding space.

3 Training

We will perform supervised fine-tuning (SFT) on the Qwen3-0.6B-Base model using a masked causal language modeling objective. The training data consists of a diverse mixture of mathematical, scientific, and coding datasets (Li et al., 2023; Zhang, 2024; Azure99, 2024; Kashani, 2024; Kirouane, 2024)

3.1 DPO

We adapted the DPO implementation from (Allam, 2023), using either a LogSigmoid-based or IPO loss to align preferences. Due to the small model size, LoRA was disabled. Training was performed in bfloat16 (as float32 caused OOM and float16 produced NaNs), using AdamW. Metrics logged include loss, reward accuracy, reward margin, KL

divergence (for both completions), and average relative log-probabilities relative to the frozen reference model.

3.2 MCQA

Due to computational constraints, we conducted preliminary training on a small subset of MathQA data using CPU-only resources. We perform fine-tuning on the Qwen3-0.6B-Base model using masked causal language modeling. The model uses next-token prediction loss on answer explanations, with context tokens masked to focus learning on response generation.

3.3 Quantization

We explain the Quantization Aware Training (QAT) approach we used: QLoRA. The LoRA adapters learn to function optimally with the quantized base model. In the current stages, we tried two different losses:
1) Next Token Prediction Loss, focusing on generating only the correct answer.
2) Contrastive learning loss, the data were structured into positive pairs (question + correct option) and negative pairs (question + incorrect option). Each question was ensured to have one correct answer and 3 incorrect options. The objective is a margin-based contrastive loss; see Appendix B for details.

3.4 RAG

We reused the MCQA fine-tuned model for our RAG generator to leverage its domain-specific training on STEM multiple-choice questions. For the embedder, we evaluated three HuggingFace Hub models (*all-MiniLM-L12-v2*, *all-MiniLM-L6-v2* and *MathBERT*) on held-out STEM QA tests (MathQA (Amini et al., 2019), SciQ (Welbl et al., 2017) and Camel AI Math (Li et al., 2023)) and selected *all-MiniLM-L12-v2*. We froze its backbone and added a 384×384 linear adapter to refine STEM-specific embeddings (LaTeX, code, formulas). We compared Triplet Margin Loss (Schroff et al., 2015), which enforces a margin between positive and random negative pairs, and Cosine Embedding Loss (Hadsell et al., 2006), which maximizes query-answer similarity. After testing three adapter schemes (query only, joint query and chunk, and separate adapters for query and chunk), we adopted the highest-performing configuration (Schopf et al., 2023).

4 Preliminary results

4.1 Reward Model

A preliminary run using LogSigmoid loss with batch size 16, $\beta = 0.1$, sequence length 128, and learning rate 10^{-6} (under one epoch) produced reasonable but unstable results (see Appendix Fig. 1).

4.2 MCQA Model

Training uses bfloat16 precision, AdamW optimizer with learning rate 2e-5 and cosine scheduling. Initial training on a limited MathQA subset showed convergence but suboptimal performance due to insufficient data scale and CPU-only training constraints. The model didn't demonstrate improvements in accuracy on our evaluation dataset (0.49), it requires expanded training for robust STEM question-answering performance.

4.3 Quantized Model

We tried several quantization approaches, all of them were directly applied to the base pretrained model since at the time of this report we still do not have a fully finetuned MCQA model. 1) PTQ using `BitsAndBytes`, we tried both 4bits and 8bits quantization. For the former we tried both NF4 and FP4 types. Whereas 8bits employs the 8-bit zero-point quantization scheme with mixed-precision decomposition, as described in LLM.int8() (Dettmers et al., 2022). 2) As describe above we also used QLoRA. 3) Post-Training Quantization with `compressed-tensors` (via SparseML). It enables 4-bit asymmetric per-tensor integer quantization. (Singh and Alistarh, 2020). 4) Post-Training Quantization utilizing Neural Magic's llmcompressor: it first employs SmoothQuant (Xiao et al., 2024), subsequently, GPTQ is applied as a layer-wise PTQ algorithm (Frantar et al., 2023). We computed the accuracy of the proposed methods by directly quantizing the base model. Results are shown in **Table 1**.

4.4 RAG Model

Integration of RAG increased MCQA accuracy from 0.50 to 0.60. Fine tuning *all-MiniLM-L12-v2* with a query only adapter under Cosine Embedding Loss peaked at epoch 25 before slight overfitting and outperformed other setups with the fastest convergence. On all three held out sets, Hit Rate@10 rose above 0.85 and MRR@10 rose above 0.70, each metric improving by one point over the unfine tuned baseline. We therefore adopted the query

only adapter as our protocol and will continue refining it. Full results are omitted for brevity.

5 Future directions

5.1 Reward Model

Training stability may improve by increasing β, batch size, and sequence length, or by exploring alternative losses such as KTO.

5.2 MCQA Model

We will scale training to the full dataset mixture with GPU resources and extended epochs. Learning rate tuning will optimize convergence stability. Additionally, we plan to incorporate Chain-of-Thought (Wei et al., 2022) explanations generated via GPT-4 API to enhance the model's complex reasoning capabilities on multi-step mathematical problems.

5.3 Quantized Model

Apply quantization to MCQA model, come up with a strategy to select the "best" quantization approach based on model size, inference speed and model accuracy. Stabilize training for QAT methods, in particular in the constrastive approach.

5.4 RAG Model

We will apply LoRA and prefix tuning to the embedder, employ multitask contrastive learning on LaTeX proofs, code snippets, and STEM lecture notes to align embeddings with mathematical notation and programming constructs, and augment our index with OpenStax textbooks and ArXiv preprints to broaden coverage and improve retrieval diversity.

References

Ahmed Allam. 2023. Direct preference optimization. https://github.com/0xallam/Direct-Preference-Optimization. Accessed: 2025-05-28.

Aida Amini, Saadia Gabriel, Peter Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 2357–2367. Association for Computational Linguistics.

Azure99. 2024. Stack overflow question-answer dataset top 300k. https://huggingface.co/datasets/Azure99/stackoverflow-qa-top-300k. Accessed: 2025.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Llm.int8(): 8-bit matrix multiplication for transformers at scale.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms.

Wikimedia Foundation. Wikimedia downloads.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. Gptq: Accurate post-training quantization for generative pre-trained transformers.

Raia Hadsell, Sumit Chopra, and Yann LeCun. 2006. Dimensionality reduction by learning an invariant mapping. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1735–1742.

Shlomo Kashani. 2024. Quantumllminstruct: A 500k llm instruction-tuning dataset with problem-solution pairs for quantum computing.

Ayoub Kirouane. 2024. Arxiv physics question-answer dataset. https://huggingface.co/datasets/ayoubkirouane/arxiv-physics. Accessed: 2025.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for "mind" exploration of large language model society. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Jannis Matas, Jonas Scheurer, and Julian Hofmann. 2021. Stackexchange title and best voted answer dataset. Accessed: 2025-05-04.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. Gpqa: A graduate-level google-proof qa benchmark.

Tim Schopf, Dennis N. Schneider, and Florian Matthes. 2023. Efficient domain adaptation of sentence embeddings using adapters. In *Proceedings of the Conference Recent Advances in Natural Language Processing - Large Language Models for Natural Language Processings*, RANLP, page 1046–1053. INCOMA Ltd., Shoumen, BULGARIA.

Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, page 815–823.

Sidak Pal Singh and Dan Alistarh. 2020. Woodfisher: Efficient second-order approximation for neural network compression.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.

Johannes Welbl, Nelson F. Liu, and Matt Gardner. 2017. Crowdsourcing multiple choice science questions. *arXiv preprint arXiv:1707.06209*.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2024. Smoothquant: Accurate and efficient post-training quantization for large language models.

Yifan Zhang. 2024. Stackmathqa: A curated collection of 2 million mathematical questions and answers sourced from stack exchange.

A Appendix

Table 1: Comparison of Quantization Techniques Applied to the Base Model (Accuracy is computed on the MMLU benchmark

Quantization Technique	Weight Quantization	Activation Quantization	Type	Accuracy
BitsAndBytes (NF4)	4-bit (NormalFloat4)	FP16/BF16 Compute	PTQ	0.439
BitsAndBytes (FP4)	4-bit (FloatPoint4)	FP16/BF16 Compute	PTQ	0.426
LLM.int8()	8-bit	8-bit/BF16/FP16	PTQ	0.520
`compressed-tensors` (via SparseML)	4-bit (Asymmetric, Per-tensor)	FP16/BF16 Compute	PTQ	0.212
`llmcompressor` (SmoothQuant + GPTQ)	8-bit	8-bit	PTQ	0.503
QLoRA (Contrastive Loss)	4-bit (NF4)	FP16/BF16 Compute	ATQ	0.415
QLoRA (NTP Loss)	4-bit (NF4)	FP16/BF16 Compute	ATQ	0.422

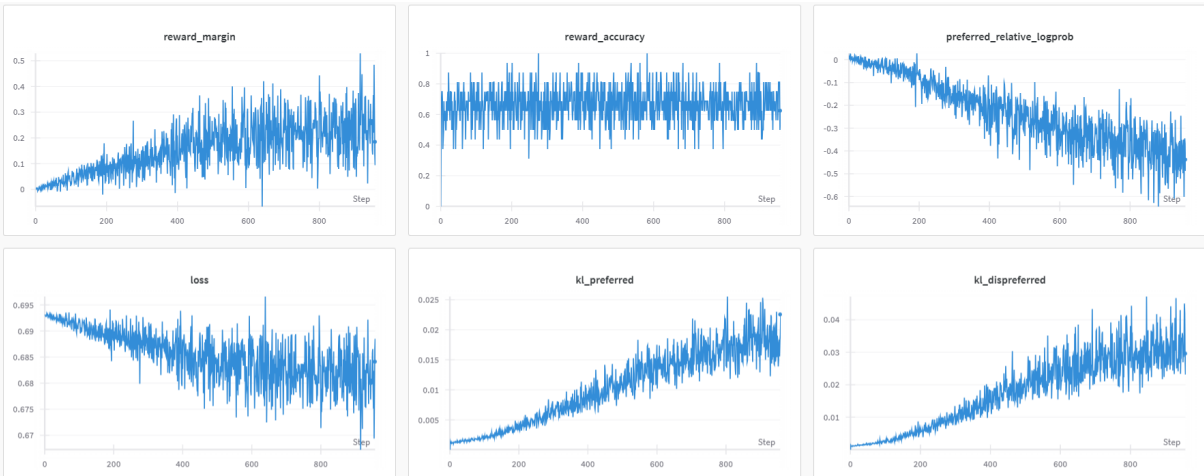

Figure 1: DPO first run

B Contrastive Loss Derivation

Let Q be the question, O_{pos} the correct option, and $O_{\text{neg},k}$ the kth incorrect option. The contrastive loss for a single positive/negative pair $(O_{\text{pos}}, O_{\text{neg},k})$ with margin m is:

$$\mathcal{L}_{\text{contrastive},k} = \max\big(0,\ m + \text{NLL}(O_{\text{pos}} \mid Q;\theta)\ -\ \text{NLL}(O_{\text{neg},k} \mid Q;\theta)\big)$$

where $\text{NLL}(O \mid Q;\theta)$ is the negative log-likelihood of option O given thr question Q. The total loss for each example is the sum over all incorrect options. The loss of the batch is the average across all the examples in the batch.